

Kotlin & Jetpack Compose Online Store - Full Guide

A beginner's guide for Flutter/Dart developers

Every concept is explained with Flutter/Dart side-by-side comparisons.

Architecture mirrors the clean architecture you already know.

All code is production-style but kept simple for learning.

Table of Contents

1. Project Setup & Dependencies
2. Clean Architecture Overview
3. Dependency Injection with Hilt
4. Models (data class)
5. Repository Pattern (Data Layer)
6. State Management (Sealed Classes + ViewModel)
7. Navigation
8. Jetpack Compose UI Basics
9. Feature: Products (Listing, Search, Categories)
10. Feature: Product Detail
11. Feature: Cart
12. Feature: Order
13. How Everything Connects
14. Flutter vs Kotlin/Compose Cheat Sheet
15. Common Beginner Questions

1. Project Setup & Dependencies

What we added to the project

A fresh Android Studio project only includes the basics. We added four key libraries to make this app work properly. Here is what each one does:

Hilt - Dependency Injection

Hilt is Google's official DI library for Android. It automatically creates and provides objects (like repositories) wherever they are needed. Without it, you would have to manually pass objects through constructors everywhere - very tedious in large apps.

[NOTE] Flutter equivalent: manually wiring dependencies through `DependenciesScope.of(context)`, but Hilt does this automatically using annotations.

Navigation Compose

Jetpack Navigation handles moving between screens. It provides a `NavController` that pushes and pops screens from a back stack, just like Flutter's Navigator.

ViewModel (lifecycle-viewmodel-compose)

ViewModel survives screen rotations. If you store state inside a regular Composable function, it is destroyed and recreated whenever the screen rotates. ViewModel keeps that state alive.

KSP - Kotlin Symbol Processing

KSP is a code generator tool. Hilt uses it to read your `@HiltViewModel` and `@Inject` annotations at compile time and generate the wiring code automatically. You never write or edit the generated files - just add annotations.

libs.versions.toml

Android projects use a Version Catalog (`libs.versions.toml`) to manage all dependency versions in one place. This prevents version conflicts. Think of it like `pubspec.yaml` in Flutter.

```
[versions]
hilt          = "2.51.1"
ksp           = "2.0.21-1.0.28"
navigationCompose = "2.8.4"

[libraries]
hilt-android          = { group = "com.google.dagger", name = "hilt-android",          version.ref = "hilt" }
hilt-android-compiler = { group = "com.google.dagger", name = "hilt-android-compiler", version.ref = "hilt" }
hilt-navigation-compose = { group = "androidx.hilt",    name = "hilt-navigation-compose", version.ref = "hilt" }
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }

[plugins]
hilt-android = { id = "com.google.dagger.hilt.android", version.ref = "hilt" }
ksp          = { id = "com.google.devtools.ksp",        version.ref = "ksp" }
```

app/build.gradle.kts

The app's build file applies the plugins and declares the dependencies:

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
    alias(libs.plugins.kotlin.compose)  
    alias(libs.plugins.hilt.android)    // enables Hilt DI  
    alias(libs.plugins.ksp)            // enables code generation  
}  
  
dependencies {  
    implementation(libs.hilt.android)  
    implementation(libs.hilt.navigation.compose)  
    ksp(libs.hilt.android.compiler)    // ksp = compile-time code generation  
    implementation(libs.androidx.navigation.compose)  
    implementation(libs.androidx.lifecycle.viewmodel.compose)  
}
```

2. Clean Architecture Overview

This project follows the same clean architecture you use in Flutter. Each feature is a self-contained folder with four layers. The dependency always flows inward - the UI layer knows about the ViewModel, but the repository knows nothing about the UI.

Folder structure (one feature)

```
features/
+-- products/
  +-- models/
    |   +-- Product.kt          <- data class (immutable model)
    |   +-- Category.kt        <- enum class
  +-- data/
    |   +-- IProductRepository.kt <- interface (contract)
    |   +-- ProductRepositoryImpl.kt <- implementation
  +-- controller/
    |   +-- ProductsState.kt    <- sealed class (all possible states)
    |   +-- ProductsViewModel.kt <- business logic + state
  +-- widgets/
    +-- ProductsScreen.kt      <- main screen composable
    +-- components/
      +-- ProductCard.kt
      +-- CategoryChips.kt
      +-- SearchBarWidget.kt
```

The four layers explained

models/ - Data Layer

Pure Kotlin data classes. No Android dependencies, no logic. They just hold data. Equivalent to your `@immutable` Flutter models.

data/ - Repository Layer

The repository talks to external data sources (API, database, etc.). In this app it uses fake in-memory data, but in a real app this is where Retrofit HTTP calls or Room DB queries live. Always defined as an interface (`IProductRepository`) so the ViewModel never depends on the concrete implementation.

controller/ - Business Logic Layer

The ViewModel holds the state and processes user actions. It calls repository methods, transforms results into states, and exposes them via `StateFlow`. The UI observes `StateFlow` and redraws when it changes.

widgets/ - Presentation Layer

Composable functions that render the UI. They receive callbacks for user actions and read state from the ViewModel. They never call repositories directly.

[NOTE] Rule of thumb: data flows DOWN (ViewModel -> UI), events flow UP (UI -> ViewModel via callbacks). This is identical to the unidirectional data flow in Flutter.

3. Dependency Injection with Hilt

Hilt removes the boilerplate of manually creating and passing objects. You just annotate classes and Hilt figures out how to build them.

Step 1: @HiltAndroidApp on the Application class

Every Hilt app needs an Application class marked with @HiltAndroidApp. This is the root of Hilt's component hierarchy. It must also be registered in AndroidManifest.xml.

```
// App.kt
@HiltAndroidApp
class App : Application()

// AndroidManifest.xml
<application android:name=".App" ...>
```

Step 2: @AndroidEntryPoint on MainActivity

Any Activity that uses Hilt must be annotated. This allows Hilt to inject dependencies into the Activity (and the Composables hosted inside it).

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() { ... }
```

Step 3: @Inject on repository constructors

When Hilt sees @Inject constructor(), it knows how to create that class automatically. No factory or builder needed.

```
class ProductRepositoryImpl @Inject constructor() : IProductRepository {
    // Hilt creates this class - no manual instantiation needed
}
```

Step 4: AppModule - binding interfaces to implementations

Since ViewModels ask for IProductRepository (the interface), Hilt needs to know which concrete class to provide. The @Binds annotation in AppModule tells it.

```
@Module
@InstallIn(SingletonComponent::class) // lives for the entire app lifetime
abstract class AppModule {

    @Binds
    @Singleton // only one instance ever created
    abstract fun bindProductRepository(impl: ProductRepositoryImpl): IProductRepository

    @Binds
    @Singleton
    abstract fun bindCartRepository(impl: CartRepositoryImpl): ICartRepository
```

```
}
```

Step 5: @HiltViewModel + @Inject on ViewModels

ViewModels get their dependencies injected automatically. Hilt reads the constructor, finds the matching bindings, and passes them in.

```
@HiltViewModel
class ProductsViewModel @Inject constructor(
    private val productRepository: IProductRepository, // Hilt injects this
) : ViewModel() { ... }
```

Step 6: hiltViewModel() in Composables

In Compose screens, call `hiltViewModel()` instead of creating the ViewModel manually. Hilt finds the existing ViewModel or creates a new one with all dependencies injected.

```
@Composable
fun ProductsScreen(
    viewModel: ProductsViewModel = hiltViewModel(), // automatic DI
) { ... }
```

[NOTE] Flutter parallel: Hilt is like DependenciesScope + constructor injection combined. In Flutter you write: final deps = DependenciesScope.of(context). Hilt does that wiring automatically at compile time using annotations.

4. Models (data class)

Kotlin's data class is the direct equivalent of an `@immutable` class in Dart. It gives you `equals()`, `hashCode()`, `toString()`, and `copy()` for free - just like Dart gives you `==` comparison and `copyWith()`.

Product model

```
// Kotlin
data class Product(
    val id: Int,
    val name: String,
    val description: String,
    val price: Double,
    val category: Category,
    val rating: Float = 0f,           // default value, same as Dart's optional named param
    val reviewCount: Int = 0,
)
```

```
// Dart equivalent
@immutable
class Product {
  const Product({
    required this.id,
    required this.name,
    required this.description,
    required this.price,
    required this.category,
    this.rating = 0.0,
    this.reviewCount = 0,
  });
  final int id;
  ...
}
```

Copying with changes

```
// Kotlin - built-in copy() on every data class
val updated = product.copy(name = "New Name", price = 49.99)

// Dart - manual copyWith() you write yourself
final updated = product.copyWith(name: "New Name", price: 49.99);
```

Category enum

Kotlin enums can have properties, just like Dart enums. The `displayName` is used in the UI chips to show a human-readable label.

```
enum class Category(val displayName: String) {
```

```
ALL("All"),
ELECTRONICS("Electronics"),
CLOTHING("Clothing"),
BOOKS("Books"),
HOME("Home"),
}
```

```
// Usage:
```

```
val label = Category.ELECTRONICS.displayName // "Electronics"
val allValues = Category.entries             // [ALL, ELECTRONICS, ...]
```

CartItem - computed property

```
data class CartItem(
    val product: Product,
    val quantity: Int,
) {
    // A getter (computed property) - not stored, always derived
    val totalPrice: Double get() = product.price * quantity
    // Dart: double get totalPrice => product.price * quantity;
}
```

5. Repository Pattern (Data Layer)

The repository is the single source of truth for data. It abstracts where the data comes from (network, database, cache, fake) so that the ViewModel never needs to know.

Interface first

```
interface IProductRepository {
    // suspend = this function can pause (for network/disk I/O) without blocking the thread
    suspend fun getProducts(): List<Product>
    suspend fun getProductById(id: Int): Product?    // ? means nullable (can return null)
    suspend fun searchProducts(query: String, category: Category): List<Product>
}
```

[NOTE] suspend in Kotlin = async in Dart. Both allow the function to wait for I/O without freezing the UI.

Implementation with fake data

```
class ProductRepositoryImpl @Inject constructor() : IProductRepository {

    private val fakeProducts = listOf(
        Product(1, "Laptop Pro", "Powerful laptop...", 999.0, Category.ELECTRONICS, 4.5f, 120),
        Product(2, "Smartphone X", "Latest flagship...", 699.0, Category.ELECTRONICS, 4.3f, 85),
        // ... more products
    )

    override suspend fun getProducts(): List<Product> = fakeProducts

    override suspend fun getProductById(id: Int): Product? =
        fakeProducts.find { it.id == id }    // returns null if not found

    override suspend fun searchProducts(query: String, category: Category): List<Product> {
        return fakeProducts.filter { product ->
            val matchesQuery = query.isBlank() ||
                product.name.contains(query, ignoreCase = true)
            val matchesCategory = category == Category.ALL || product.category == category
            matchesQuery && matchesCategory    // last expression = return value
        }
    }
}
```

Cart repository - synchronous + @Singleton

The cart stores data in memory. No network calls needed, so no suspend keyword. The @Singleton annotation ensures there is only one cart instance - if you add a product on the detail screen, the cart screen sees the same data.

```
@Singleton
class CartRepositoryImpl @Inject constructor() : ICartRepository {
```

```
private val _cartItems = mutableListOf<CartItem>()

override fun addToCart(product: Product) {
    val existing = _cartItems.find { it.product.id == product.id }
    if (existing != null) {
        val index = _cartItems.indexOf(existing)
        _cartItems[index] = existing.copy(quantity = existing.quantity + 1)
    } else {
        _cartItems.add(CartItem(product = product, quantity = 1))
    }
}

override fun getCartTotal(): Double = _cartItems.sumOf { it.totalPrice }
}
```

6. State Management (Sealed Classes + ViewModel)

Sealed classes - all possible states

A sealed class limits all subclasses to a known set. The Kotlin compiler then enforces that you handle every case in a `when()` block - no forgetting an error state.

```
// Kotlin sealed class
sealed class ProductsState {
    object Initial : ProductsState() // object = singleton, no data needed
    object Loading : ProductsState()
    data class Error(val message: String) : ProductsState()
    data class Completed(
        val products: List<Product>,
        val searchQuery: String = "",
        val selectedCategory: Category = Category.ALL,
    ) : ProductsState()
}
```

```
// Dart/Freezed equivalent
@freezed
sealed class ProductsState with _$ProductsState {
    const factory ProductsState.initial() = Products$InitialState;
    const factory ProductsState.loading() = Products$LoadingState;
    const factory ProductsState.error(String message) = Products$errorState;
    const factory ProductsState.completed({required List<Product> products}) = Products$CompletedState;
}
```

ViewModel - StateFlow

`MutableStateFlow` holds the current state. `StateFlow` is the read-only version exposed to the UI. Composables call `collectAsState()` to subscribe - they automatically redraw when the state changes.

```
@HiltViewModel
class ProductsViewModel @Inject constructor(
    private val productRepository: IProductRepository,
) : ViewModel() {

    // _state is private and mutable (only this class can change it)
    private val _state = MutableStateFlow<ProductsState>(ProductsState.Initial)

    // state is public and read-only (UI can observe but not change it)
    val state: StateFlow<ProductsState> = _state.asStateFlow()

    fun load(query: String = "", category: Category = Category.ALL) {
        // viewModelScope = coroutine scope tied to ViewModel lifetime
        viewModelScope.launch {
            _state.value = ProductsState.Loading
            try {
```

```

        val products = productRepository.searchProducts(query, category)
        _state.value = ProductsState.Completed(products = products)
    } catch (e: Exception) {
        _state.value = ProductsState.Error(e.message ?: "Unknown error")
    }
}
}
}

```

Observing state in Compose

```

@Composable
fun ProductsScreen(viewModel: ProductsViewModel = hiltViewModel()) {

    // collectAsState() subscribes to the flow.
    // The composable redraws automatically whenever _state.value changes.
    val state by viewModel.state.collectAsState()

    when (val currentState = state) {
        is ProductsState.Loading -> CircularProgressIndicator()

        is ProductsState.Error -> Text(currentState.message) // smart cast: knows it's Error

        is ProductsState.Completed -> {
            // smart cast: knows it's Completed, can access .products directly
            LazyVerticalGrid(...) {
                items(currentState.products) { product ->
                    ProductCard(product = product, onClick = { ... })
                }
            }
        }

        is ProductsState.Initial -> { /* nothing */ }
    }
}

```

[NOTE] `collectAsState()` = `ListenableBuilder` in Flutter. Both re-render the widget/composable whenever the underlying state changes.

7. Navigation

Navigation Compose uses a `NavHost` to define all routes and a `NavController` to navigate between them. Routes are plain strings (like URL paths).

AppRoute - all routes in one place

```
sealed class AppRoute(val route: String) {
    object Products : AppRoute("products")
    object Cart : AppRoute("cart")
    object Order : AppRoute("order")

    // Route with a path parameter (like /product_detail/42)
    object ProductDetail : AppRoute("product_detail/{productId}") {
        fun createRoute(productId: Int) = "product_detail/$productId"
    }
}
```

AppNavigation - the NavHost

```
@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = AppRoute.Products.route) {

        composable(AppRoute.Products.route) {
            ProductsScreen(
                onProductClick = { id -> navController.navigate(AppRoute.ProductDetail.createRoute(id)) },
                onCartClick = { navController.navigate(AppRoute.Cart.route) },
            )
        }

        composable(
            route = AppRoute.ProductDetail.route,
            arguments = listOf(navArgument("productId") { type = NavType.IntType }),
        ) { backStackEntry ->
            val productId = backStackEntry.arguments?.getInt("productId") ?: return@composable
            ProductDetailScreen(productId = productId, ...)
        }

        composable(AppRoute.Order.route) {
            OrderScreen(
                onContinueShoppingClick = {
                    // popUpTo clears the back stack so Back doesn't go to the order screen
                    navController.navigate(AppRoute.Products.route) {
                        popUpTo(AppRoute.Products.route) { inclusive = true }
                    }
                },
            ),
        }
    }
}
```

```
        )  
    }  
}  
}
```

[NOTE] Flutter parallel:

navController.navigate(route) = Navigator.pushNamed(context, route)

navController.popBackStack() = Navigator.pop(context)

popUpTo(...) = Navigator.pushAndRemoveUntil(...)

8. Jetpack Compose UI Basics

Jetpack Compose describes UI as functions (not XML layouts). A `@Composable` function is called repeatedly to redraw the screen when state changes - just like Flutter's `build()`.

Key Compose concepts vs Flutter

Flutter (Dart)	Compose (Kotlin)
<code>Widget build(BuildContext ctx)</code>	<code>@Composable fun MyWidget()</code>
<code>StatelessWidget</code>	<code>@Composable</code> function (no state)
<code>setState() { ... }</code>	<code>mutableStateOf()</code> + <code>remember {}</code>
<code>ListenableBuilder</code>	<code>collectAsState()</code> on <code>StateFlow</code>
<code>Column / Row / Stack</code>	<code>Column / Row / Box</code>
<code>ListView.builder</code>	<code>LazyColumn { items(...) }</code>
<code>GridView.builder</code>	<code>LazyVerticalGrid { items(...) }</code>
<code>Padding widget</code>	<code>Modifier.padding(...)</code>
<code>SizedBox / Spacer</code>	<code>Spacer(modifier = Modifier.height(8.dp))</code>
<code>GestureDetector / InkWell</code>	<code>Modifier.clickable { ... }</code>
<code>initState()</code>	<code>LaunchedEffect(Unit) { ... }</code>
<code>Scaffold + AppBar</code>	<code>Scaffold { topBar = { TopAppBar() } }</code>
<code>CircularProgressIndicator()</code>	<code>CircularProgressIndicator()</code>
<code>ElevatedButton</code>	<code>Button(onClick = { ... }) { Text(...) }</code>

remember and mutableStateOf

`remember` keeps a value alive across recompositions (redraws). `mutableStateOf` creates a value that triggers a redraw when changed. Together they replace `StatefulWidget`'s local state.

```
// Compose
var searchQuery by remember { mutableStateOf("") }
var selectedCategory by remember { mutableStateOf(Category.ALL) }

// Flutter equivalent
String searchQuery = "";
Category selectedCategory = Category.all;
// ... inside setState() to trigger rebuild
```

LaunchedEffect - running code once on entry

```
// Runs once when the composable enters the tree (Unit as key = run only once)
LaunchedEffect(Unit) {
    viewModel.load()
}

// Runs whenever productId changes
```

```
LaunchedEffect(productId) {  
    viewModel.load(productId)  
}
```

```
// Flutter equivalent: initState() for once, or didUpdateWidget() for change
```

Modifier - the layout system

Modifier chains describe how a composable is sized, padded, and interacted with. Order matters - padding before clickable is different from clickable before padding.

```
Text(  
    text = "Hello",  
    modifier = Modifier  
        .fillMaxWidth()           // take all available width  
        .padding(16.dp)           // add space inside  
        .clickable { ... }        // make it tappable  
)
```

9. Feature: Products

The products feature shows a scrollable 2-column grid of all products. Users can type in a search bar to filter by name/description, and tap category chips to filter by category. Both filters can be combined.

How search + category filtering works

The `SearchBarWidget` and `CategoryChips` are stateless - they receive their current value and an `onChanged` callback. The `ProductsScreen` owns the local state and calls `viewModel.load()` whenever either changes.

```
// ProductsScreen.kt
var searchQuery by remember { mutableStateOf("") }
var selectedCategory by remember { mutableStateOf(Category.ALL) }

SearchBarWidget(
    query = searchQuery,
    onQueryChange = { query ->
        searchQuery = query
        viewModel.load(query = query, category = selectedCategory)
    },
)

CategoryChips(
    selectedCategory = selectedCategory,
    onCategorySelected = { category ->
        selectedCategory = category
        viewModel.load(query = searchQuery, category = category)
    },
)
```

Filtering logic in the repository

```
// ProductRepositoryImpl.kt
override suspend fun searchProducts(query: String, category: Category): List<Product> {
    return fakeProducts.filter { product ->
        val matchesQuery = query.isBlank() || // empty query = show all
            product.name.contains(query, ignoreCase = true) ||
            product.description.contains(query, ignoreCase = true)
        val matchesCategory = category == Category.ALL || product.category == category
        matchesQuery && matchesCategory // both must be true
    }
}
```

LazyVerticalGrid - 2-column product grid

```
LazyVerticalGrid(
    columns = GridCells.Fixed(2), // always 2 columns
    contentPadding = PaddingValues(16.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp), // gap between columns
)
```

```
verticalArrangement = Arrangement.spacedBy(8.dp),    // gap between rows
) {
  items(currentState.products, key = { it.id }) { product ->
    // key = stable ID for efficient recomposition (like Flutter's Key)
    ProductCard(product = product, onClick = { onProductClick(product.id) })
  }
}
```

10. Feature: Product Detail

The product detail screen loads a product by ID and lets the user add it to the cart. Once added, the button disables itself to prevent accidental double-adds.

Loading by ID from navigation argument

```
// ProductDetailScreen.kt
LaunchedEffect(productId) {
    viewModel.load(productId) // load when the screen appears (or productId changes)
}
```

Add to cart logic in ViewModel

```
// ProductDetailViewModel.kt
fun addToCart(product: Product) {
    cartRepository.addToCart(product)

    // Smart cast: after `is Completed` check, Kotlin knows the exact type
    val current = _state.value
    if (current is ProductDetailState.Completed) {
        _state.value = current.copy(addedToCart = true) // disable the button
    }
}
```

Button disabled after adding

```
Button(
    onClick = { viewModel.addToCart(product) },
    modifier = Modifier.fillMaxWidth(),
    enabled = !currentState.addedToCart, // disabled once added
) {
    Text(
        text = if (currentState.addedToCart) "Added to Cart (done)" else "Add to Cart",
    )
}
```

11. Feature: Cart

The cart feature displays all cart items with quantity controls. The total is shown in the bottom bar with a Checkout button. The cart data lives in a `@Singleton` repository so it persists across screen navigations.

Why CartRepositoryImpl is @Singleton

Without `@Singleton`, every time a `ViewModel` asks for `ICartRepository`, Hilt would create a new empty `CartRepositoryImpl`. The `@Singleton` annotation tells Hilt to reuse the same instance - so adding to the cart on the detail screen is visible on the cart screen.

Quantity controls

```
// CartViewModel.kt
fun increaseQuantity(productId: Int) {
    val item = cartRepository.getCartItems().find { it.product.id == productId }
    if (item != null) {
        cartRepository.updateQuantity(productId, item.quantity + 1)
        load() // reload state to trigger UI update
    }
}

fun decreaseQuantity(productId: Int) {
    val item = cartRepository.getCartItems().find { it.product.id == productId }
    if (item != null) {
        cartRepository.updateQuantity(productId, item.quantity - 1)
        // updateQuantity removes the item if quantity drops to 0
        load()
    }
}
```

Bottom bar with total

```
// CartScreen.kt
bottomBar = {
    if (state is CartState.Completed) {
        val completedState = state as CartState.Completed
        Surface(tonalElevation = 3.dp) {
            Row(modifier = Modifier.fillMaxWidth().padding(16.dp),
                horizontalArrangement = Arrangement.SpaceBetween) {
                Column {
                    Text("Total")
                    Text(text = "$${completedState.total}", fontWeight = FontWeight.Bold)
                }
                Button(onClick = onCheckoutClick) { Text("Checkout") }
            }
        }
    }
}
```

12. Feature: Order

The order screen is triggered when the user taps Checkout on the cart screen. It immediately calls `placeOrder()`, shows a loading spinner (simulating a network request), then displays a confirmation with all ordered items.

Simulated network delay

```
// OrderRepositoryImpl.kt
override suspend fun placeOrder(items: List<CartItem>): Order {
    delay(1500) // simulate a 1.5 second API call
    val total = items.sumOf { it.totalPrice }
    return Order(
        id = orderIdCounter++,
        items = items,
        totalPrice = total,
        status = OrderStatus.CONFIRMED,
    )
}
```

placeOrder in ViewModel

```
// OrderViewModel.kt
fun placeOrder() {
    val items = cartRepository.getCartItems()
    if (items.isEmpty()) {
        _state.value = OrderState.Error("Cart is empty")
        return
    }
    viewModelScope.launch {
        _state.value = OrderState.Loading
        try {
            val order = orderRepository.placeOrder(items)
            cartRepository.clearCart() // empty the cart after success
            _state.value = OrderState.Completed(order = order)
        } catch (e: Exception) {
            _state.value = OrderState.Error(e.message ?: "Unknown error")
        }
    }
}
```

Clearing the back stack after order

```
// AppNavigation.kt - after order, go back to products and clear the back stack
onContinueShoppingClick = {
    navController.navigate(AppRoute.Products.route) {
        popUpTo(AppRoute.Products.route) { inclusive = true }
        // Without this, pressing Back from Products would go to the order screen
    }
}
```

}

13. How Everything Connects

Here is the full data flow for 'user searches for a product':

```

User types "laptop" in SearchBarWidget
    |
    v
onQueryChange callback fires in ProductsScreen
    |
    v
viewModel.load(query = "laptop", category = ALL)
    |
    v
ProductsViewModel.load() sets state = Loading, then calls:
    |
    v
productRepository.searchProducts("laptop", Category.ALL)
    |
    v
ProductRepositoryImpl filters fakeProducts list
    |
    v
Returns List<Product> (e.g. [Laptop Pro])
    |
    v
ViewModel sets state = Completed(products = [Laptop Pro])
    |
    v
StateFlow emits new value
    |
    v
collectAsState() triggers recomposition in ProductsScreen
    |
    v
LazyVerticalGrid renders one ProductCard
  
```

And for 'user adds product to cart and checks out':

```

ProductDetailScreen: user taps "Add to Cart"
    |
    v
viewModel.addToCart(product) called
    |
    v
CartRepositoryImpl.addToCart() - adds/increments in-memory list
    |
    v
ProductDetailState.Completed.copy(addedToCart = true)
    |
    v
Button becomes disabled, text changes to "Added to Cart (done)"
  
```

```
|
v
User taps cart icon -> navigates to CartScreen
|
v
LaunchedEffect(Unit) -> cartViewModel.load()
|
v
CartRepositoryImpl.getCartItems() returns the same list (Singleton!)
|
v
CartState.Completed rendered with items + total
|
v
User taps Checkout -> navigates to OrderScreen
|
v
LaunchedEffect(Unit) -> orderViewModel.placeOrder()
|
v
Shows Loading spinner for 1.5s (delay simulation)
|
v
CartRepositoryImpl.clearCart() called
|
v
OrderState.Completed -> confirmation screen shown
```

14. Flutter vs Kotlin/Compose Cheat Sheet

Flutter / Dart Concept	Kotlin / Jetpack Compose Equivalent
pubspec.yaml	libs.versions.toml + build.gradle.kts
@immutable class + copyWith()	data class + .copy()
@freezed sealed class	sealed class with object/data class
abstract interface class IRepo	interface IRepo
StateController<State>	ViewModel + MutableStateFlow<State>
setState(const State.loading())	_state.value = State.Loading
ListenableBuilder	collectAsState() + recomposition
DependenciesScope.of(context)	hiltViewModel() with Hilt DI
Constructor injection	@Inject constructor + Hilt
@Singleton via DI setup	@Singleton + @Binds in @Module
async/await	suspend + coroutines
Stream / StreamController	Flow / MutableStateFlow
Widget build(ctx)	@Composable fun MyScreen()
StatelessWidget	@Composable function
StatefulWidget + setState	remember + mutableStateOf
initState()	LaunchedEffect(Unit) { }
didUpdateWidget()	LaunchedEffect(key) { } with key
dispose()	ViewModel.onCleared() / DisposableEffect
Column / Row / Stack	Column / Row / Box
ListView.builder	LazyColumn { items(...) }
GridView.builder	LazyVerticalGrid { items(...) }
SizedBox(height: 8)	Spacer(Modifier.height(8.dp))
Padding widget	Modifier.padding(...)
GestureDetector / InkWell	Modifier.clickable { }
Scaffold + AppBar	Scaffold { topBar = { TopAppBar() } }
CircularProgressIndicator()	CircularProgressIndicator()
ElevatedButton / TextButton	Button / TextButton
Card widget	Card { }
Text widget	Text(text = ...)
Navigator.push()	navController.navigate(route)
Navigator.pop()	navController.popBackStack()
pushAndRemoveUntil()	navigate { popUpTo(...) }
GoRouter path parameters	navArgument("id") { type = NavType.IntType }
context.screenSizeMaybeWhen	WindowSizeClass (separate library)
dart run build_runner build	Gradle sync (automatic with KSP)

15. Common Beginner Questions

Q1: What is the difference between object and data class in a sealed class?

object is a singleton with no data - used for states like Loading or Initial that carry no extra information. data class has fields - used for states like Completed or Error that carry data. You cannot call `.copy()` on an object.

Q2: Why does the ViewModel use MutableStateFlow and expose StateFlow?

Keeping the mutable version private prevents any Composable from directly changing state. Only the ViewModel can call `_state.value = ...`, keeping state changes predictable and traceable. The public StateFlow is read-only - the UI can observe but not mutate.

Q3: What is the difference between suspend fun and a regular fun?

suspend means the function can pause at any point (e.g., waiting for a network response) and resume later without blocking the thread. Regular functions run synchronously from start to finish. You must call suspend functions from a coroutine scope (`viewModelScope.launch`) or another suspend function.

Q4: Why is CartRepositoryImpl @Singleton but ProductRepositoryImpl is not?

CartRepositoryImpl holds mutable in-memory state (the cart list). If two different instances existed, they would have different lists - adding a product in one would not appear in the other. ProductRepositoryImpl only has read-only fake data, so multiple instances would not be a problem (though Singleton is fine there too).

Q5: What does 'smart cast' mean?

After you check 'if (x is Completed)', Kotlin knows x is of type Completed inside that block - you can access `currentState.products` without casting. In Java or older Kotlin you would write `((Completed) x).products`. Smart cast removes that noise.

Q6: What is the difference between LaunchedEffect(Unit) and LaunchedEffect(key)?

LaunchedEffect(Unit) runs once when the composable enters the composition, like `initState()` in Flutter. LaunchedEffect(productId) runs again every time productId changes. Use the second form when you need to reload data based on a changing parameter.

Q7: Why does AppModule use @Binds and not @Provides?

@Binds is used when you just want to map an interface to an implementation that already has @Inject constructor. It generates less code than @Provides. Use @Provides when you need to manually construct an object (e.g., Retrofit, OkHttpClient) that you cannot annotate with @Inject.

Q8: How do I add real images instead of placeholder text?

Add the Coil library: `implementation('io.coil-kt:coil-compose:2.7.0')`. Then replace the Surface placeholder with:

```
AsyncImage(model = product.imageUrl, contentDescription = product.name,
    modifier = Modifier.fillMaxWidth().height(120.dp))
```

Q9: How do I replace the fake data with a real REST API?

Add Retrofit: `implementation('com.squareup.retrofit2:retrofit:2.11.0')`. Create a @GET annotated interface, build a Retrofit instance in AppModule using @Provides, and inject it into ProductRepositoryImpl. Your suspend fun `getProducts()` then calls the Retrofit service instead of returning fakeProducts.

Q10: What does 'by' do in 'val state by viewModel.state.collectAsState()'?

'by' is Kotlin's property delegation syntax. It means: instead of calling `viewModel.state.collectAsState().value` every time, Kotlin automatically unwraps the delegate and gives you the value directly. It is equivalent to writing `val stateObj = viewModel.state.collectAsState(); use stateObj.value`.

Quick Reference: Annotation Guide

Here is every annotation used in this project and what it does:

```
@HiltAndroidApp      -> marks Application class as Hilt root
@AndroidEntryPoint   -> enables Hilt injection in Activity/Fragment
@HiltViewModel        -> tells Hilt how to create and inject this ViewModel
@Inject constructor() -> tells Hilt it can construct this class automatically
@Module              -> this class provides dependency bindings to Hilt
@InstallIn(SingletonComponent::class) -> bindings live for the entire app lifetime
@Binds                -> maps interface -> implementation (no code needed)
@Provides             -> manually constructs an object for Hilt (e.g. Retrofit)
@Singleton            -> only one instance is ever created
@Composable           -> this function describes UI (like Widget build() in Flutter)
@OptIn(ExperimentalMaterial3Api::class) -> acknowledges use of experimental APIs
```

Quick Reference: Kotlin Syntax

```
val name: Type        // read-only (like final in Dart)
var name: Type         // mutable
Type?                  // nullable type (can be null, like Type? in Dart)
x?.method()            // null-safe call (like x?.method() in Dart)
x ?: default           // if x is null use default (like x ?? default in Dart)
listOf(a, b, c)         // immutable list
mutableListOf(a, b, c)  // mutable list
list.filter { it.active } // filter (like list.where((e) => e.active))
list.map { it.name }     // map (like list.map((e) => e.name))
list.find { it.id == id } // find first (like list.firstWhereOrNull(...))
list.sumOf { it.price }  // sum (like list.fold(0.0, (s, e) => s + e.price))
when (x) { is Foo -> ... } // pattern match (like switch() in Dart)
data class Foo(val a: Int) // immutable model with .copy()
sealed class S           // exhaustive sum type (like @freezed sealed)
object Singleton         // single instance (like a class with only static members)
enum class E(val v: String) // enum with properties
```